Technical report: Klartext

Batch triage of official Swiss letters for caseworkers, with every extracted fact checked against the letter in code.

- **Track:** Track 2B: Own Project
- **Event:** Online
- **Team:** Baguette & Baklava: Yusuf Öztoprak, Alexis Merle
- **Demo:** https://youtu.be/zZ5clDtspV0
- **Code:** https://github.com/LeFlegm/Klartext (`track_2b/`)

1. Summary

Caseworkers in social services, municipalities and legal aid offices receive piles of official letters (tax offices, health insurers, debt collection) from people who often cannot read them. The expensive part is not reading one letter, it is deciding which of fifty letters must be handled today. Klartext reads a batch of PDF letters with Apertus, extracts sender, deadline, required actions and consequences, and sorts the letters by urgency. The central design decision is that **the model is never trusted on its own**: for every fact it must return a verbatim quote from the letter, and code (not the model) checks that the quote really occurs in the letter, that a date really occurs in its quote, and that a period such as “within 30 days” contains that number. Anything that fails is shown as unverified. An 8B model answers first and a 70B model is called only when a check fails. On our ten-letter development set, checking turned all silent errors into flagged ones (0 undetected errors, against 1 of 42 for raw 8B and 3 of 41 for raw 70B), and the cascade needed about half the latency of always using 70B. These numbers come from a development set that also shaped our checks, so they show a direction, not a guarantee (Section 5).

2. Architecture

```
Browser (Next.js, static export)
   | PDF upload
   v
FastAPI container  ->  pdfplumber (text, header/footer cleanup)
   |                   Apertus 8B extraction  -> verify in code
   |                       | any point unverified or invalid JSON
   |                       v
   |                   Apertus 70B extraction -> verify in code
   |                   triage: sort by days left
   |                   explanation (70B, verified facts only, 10 languages)
   v
Apertus endpoint (OpenAI-compatible API, e.g. vLLM)
```

Components, all in one container started by `make run`:

- **PDF reading** (`adapters/pdf.py`): pdfplumber text extraction; hyphenated line breaks are joined; header and footer lines that repeat on several pages are removed, but only at the edges of a page, so a letterhead repeated in a signature survives. Scanned PDFs without a text layer are rejected with a clear error (no OCR).
- **Extraction** (`services/extractor.py`): one prompt, temperature 0, JSON output with an English `value` and a verbatim `source_span` per fact. One automatic retry shows the model its invalid output.
- **Verification** (`domain/validation.py`): normalisation (Unicode, quotes, whitespace, hyphen joins, case), then a substring check with a minimum span length. A date must appear in its own quote (`domain/dates.py` parses German, French and Italian formats), a relative period must contain its number, and a deadline that claims both a date and a period is flagged because it usually merges two deadlines. The model has no way to set `verified`.
- **Cascade** (`services/pipeline.py`): 8B first; if any point is unverified or the output is invalid, the 70B answer replaces it (`escalated: true`); if 70B also fails, the flagged 8B answer is kept.

- **Triage**: days left are computed in code. A relative deadline is counted from the letter date and shown as an estimate. Letters without a deadline come last.
- **Explanation** (`services/explainer.py`): the 70B model writes at most five short sentences in one of ten languages (English, French, German, Italian, Turkish, Arabic, Ukrainian, Tigrinya, Dari, Somali) from the verified facts only; unverified points never reach this step.
- **UI**: Next.js static export, served by the same FastAPI process. Letters and results live in memory only.

Target architecture (mandatory)

Klartext is deployable **on-premise and air-gapped** (a and b), and equally on a Swiss sovereign cloud (c).

- **Runtime dependencies:** only the Apertus endpoint, addressed by `LLM_BASE_URL`. Pointing it at an Apertus server inside the same network (for example vLLM on the organisation's own hardware) means that no data leaves the building. There is no telemetry, no third-party API, no CDN call and no database.
- **Build time vs runtime:** `pip install` and `npm ci` need internet when the image is built. The running container does not. For a fully offline site the image is built once elsewhere and transferred.
- **Data handling:** letters and results are kept in process memory and disappear when the container stops. Nothing is written to disk. All sample letters are synthetic.
- During the hackathon we used the organisers' hosted endpoint for development only.

3. Use of Apertus

- **Models:** `apertus-v1.5-8b` (first pass) and `apertus-v1.5-70b` (escalation and explanations). No other model is used, not even as a judge.
- **How it is used:** inference only, temperature 0, no fine-tuning. Extraction is a single structured-output prompt with two worked examples (a dated and a period deadline) and strict rules: copy quotes character for character, write values in English, the sender is the authority and not the addressee, never calculate dates, list only consequences the letter states. The prompt hash is stored with each benchmark run.
- **Where it runs:** the hackathon's hosted endpoint, accessed through the OpenAI-compatible API that serving stacks such as vLLM expose. The `openai` Python package is used purely as an HTTP client pointed at `LLM_BASE_URL`; no OpenAI models or services are involved.
- **Why two sizes:** the 8B model is fast and usually right; the 70B model is slower and was also wrong on some letters. Checks in code decide when the larger model is worth its cost, instead of using it for everything.
- **Why Apertus and not a general model:** the use case needs German, French and Italian official language, runs on sensitive personal letters, and must be deployable under the organisation's own control. Apertus is open, Swiss and self-hostable.

4. Data

- **Letters:** ten synthetic letters in German, French and Italian (tax, health insurance, debt enforcement), including traps such as a letterhead repeated in the signature, two dates in one letter, a period instead of a date, a sidebar that reorders text, and letters with no deadline. The layouts were designed by hand to match official Swiss formats (KVG health insurance, cantonal tax, debt enforcement). The text and the legal phrasing were drafted with LLM assistance (Gemini and Claude) and then curated by hand to add realistic visual edge cases such as manual hyphenation, sidebars, multi-page headers and QR codes.
- **Ground truth:** written by hand for each letter (sender, deadline, actions, consequences, each with a quote). A script checks that every ground-truth quote really occurs in the letter.
- **Licence and privacy:** all content is synthetic and invented by the team; no real person, address or case is included. Letters, ground truth and saved model answers are in `data/` (under 1 MB).
- **Availability:** the evaluation letters, ground truth and raw model responses are in the repository (`data/` and `data_blind/`). We did not publish a separate Hugging Face dataset.

5. Evaluation

Task. For each letter, extract sender, deadline, actions and consequences. A point is *correct* if it matches the ground truth (sender and quotes compared after normalisation, actions and consequences by word-overlap F1 of their quotes of at least 0.5, deadline by date or period). An **undetected error** is a wrong point that is displayed as verified, which is the failure that harms a caseworker. A **caught error** is a wrong point that is flagged unverified. A **false alarm** is a correct point that is flagged.

Setups. A: 8B raw. B: 70B raw. C: 8B plus checks. D: 70B plus checks. E: cascade (8B, escalating to 70B). Raw model answers are saved per letter and per model, so scoring is repeatable without API calls.

Setup	Undetected errors	Points judged	Note
A: 8B raw	1	42	no check
B: 70B raw	3	41	no check
C: 8B + checks	0	39	
D: 70B + checks	0	34	
E: cascade	0	35	about 3.3 s per letter vs 6.0 s for 70B; 3 of 10 letters escalated

Reading the table: the larger model made more silent mistakes than the smaller one on this set, so size alone does not buy safety; the checks do. The cascade kept the zero-undetected result at roughly half the latency of always calling 70B.

What this does and does not show. These ten letters were also our development set: several rules (hyphen joins, edge-only footer stripping, rejecting a deadline that claims both a date and a period) were added after seeing failures on them. We deliberately kept the checks conservative, because a false alarm costs a caseworker a quick look while a wrong but verified answer costs far more. The numbers are small and tuned, so they indicate a direction and not a measured error rate.

Blind test. Six new letters (German, French, Italian and English, one with no deadline at all) were written by a teammate after the pipeline was frozen. We ran them once and changed no code, prompt or rule afterwards. The only edit was to the format of one ground-truth file (`document_type` and the empty deadline were rewritten to match the schema); its content is unchanged.

Setup	Undetected errors	Points judged	Note
A: 8B raw	2	21	no check
B: 70B raw	1	20	no check
C: 8B + checks	1	19	a wrong payment-period quote passed the check
D: 70B + checks	0	18	
E: cascade	0	18	4.0 s per letter vs 5.6 s for 70B; 2 of 6 letters escalated

Document type, sender and deadline were correct for all six letters in every setup. Three things temper the result. First, the cascade avoided the error in C only because that letter happened to be escalated, so part of the zero is chance. Second, the system misses information: recall was 8 of 10 true points, and two payment actions in two letters were not extracted. Third, one correct point was flagged as unsupported (a false alarm) and one wrong point was flagged and not shown as verified, which is the intended behaviour. With six letters this shows that the check works, not what the error rate is. The saved raw answers and score table are in `data_blind/runs/`.

6. Limitations

- **No OCR.** Scanned letters are rejected, not guessed.

- **One deadline per letter.** Letters with several deadlines show one; a merged date-and-period deadline is flagged.
- **Layout.** Multi-column layouts and sidebars can reorder words; such quotes fail verification and are flagged instead of trusted.
- **“Verified” is not “right”.** Verified means the quote exists in the letter and supports the stated date or fact; it does not prove the model chose the right passage. A real but irrelevant fragment can pass (seen once in the blind set, in the 8B-only setup; the cascade avoided it by escalating that letter). A second check that tests whether a quote actually states an obligation is the next step.
- **Verification proves presence, not meaning.** A quote that exists in the letter can still be the wrong quote, or the English summary can misstate it. This is why the quote is always shown next to the summary for the caseworker to read.
- **Explanations** are generated text. They use only verified facts, but the wording itself is not checked.
- **State** is in memory in one process: a restart loses uploaded letters, and scaling out needs a shared store.
- **Evaluation** is small, synthetic, and partly tuned (Section 5). We did not test real letters, other cantons’ formats beyond the samples, or languages outside German, French and Italian for extraction.

7. Reproducibility

- **Run:** `cp .env.example .env`, set `LLM_API_KEY`, then `make run` from `track_2b/`. The app is at http://localhost:8000. Tested from a clean clone, with and without `.env`; without a key the API returns a readable 503.
- **Tests:** `cd src && python -m pytest` (unit tests for verification, dates, relative deadlines, PDF layout, metrics, cascade, configuration and API wiring).
- **Numbers:** `python scripts/score_benchmark.py` re-scores a saved run in `data/runs/` offline. `python scripts/run_benchmark.py` repeats the model calls (needs the endpoint).
- **Runtime:** Python 3.12 and Node 22 inside Docker; no GPU on our side, models are served remotely. Temperature 0; the hosted endpoint is not guaranteed to be bit-for-bit deterministic.
- **Commit:** the pipeline was evaluated and the app tested at commit `a9ab777`; later commits change only the report and README.

8. Next steps

- Blind evaluation on a larger set of letters from more cantons, ideally reviewed by real caseworkers.
- OCR for scanned letters, with the same verification (quotes checked against the recognised text, with a lower trust level).
- Several deadlines per letter, and recurring or instalment deadlines.
- Show the verification state prominently in the UI (unverified badge, quote next to each fact) and let the caseworker confirm or correct a fact.
- Shared storage and a queue for multi-user, multi-instance deployment, with encryption at rest if retention is ever needed.
- Quantised 8B and 70B served locally (vLLM) to measure cost per letter on on-premise hardware.

License

Following the hackathon terms (section 6): code under Apache License 2.0 (see `LICENSE` in the repository), this report and other documentation under Creative Commons Attribution 4.0 (CC-BY-4.0), and the evaluation data (letters, ground truth, raw model answers) under the Community Data License Agreement, Permissive 2.0 (CDLA-Permissive-2.0).

References

- Apertus v1.5 8B and 70B: https://huggingface.co/swiss-ai/Apertus-v1.5-8B, https://huggingface.co/swiss-ai/Apertus-v1.5-70B
- Hack Apertus: https://hackapertus.ch